

Billiards Project 2025

Documentation

Authors: Guillaume Saint-Pierre, Louis Gosset, Joseph Belamich

Section: Mechanical Engineering (GM) | **Original date:** 28 December 2025

English translation of the original French report. File names, identifiers and the documented behaviour of the original version have been retained.

1 Development environment

- Operating system and version: macOS 15.0
- Test platform: MacBook Air M1
- C compiler: Visual Studio Code (1.97.2), as stated in the original report
- LabVIEW: 2025 (25.3.2)
- MATLAB: R2025b

2 File organisation

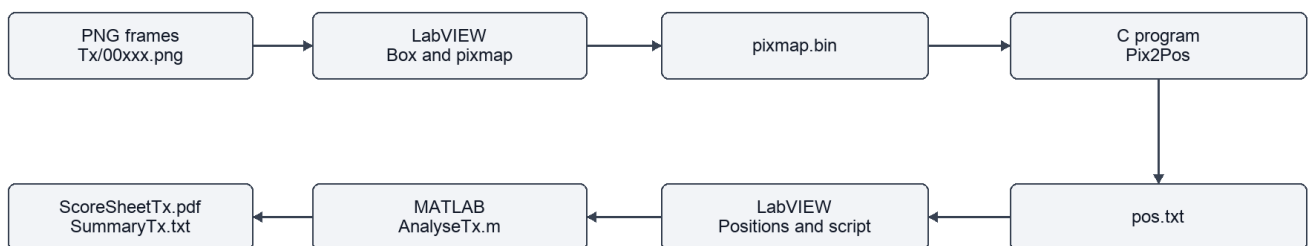
File	Role	Status
Billard2025.vi	Main VI running the complete pipeline	Supplied
FindBillardBox.vi	Detects the inner rectangle of the table	Supplied
FrameToPosition.vi	Writes pixmap.bin, runs Pix2Pos, reads pos.txt	Supplied
MatlabScript.vi	Generates the MATLAB script and inserts parameters	Supplied
RunMatlab.vi	Runs MATLAB from the command line and captures output	Supplied
Pix2Pos.c	C source for ball detection and error reporting	Supplied
Pix2Pos	Executable produced by compilation	Supplied
pixmap.bin	Binary image: W, H and 0x00RRGGBB pixels	Generated
pos.txt	Positions and scores: Red/Yellow/White: x, y, score	Generated
AnalyseTx.m	Automatically generated MATLAB analysis script	Generated
ScoreSheetTx.pdf	Score sheet in PDF format	Generated
SummaryTx.txt	Compact text summary	Generated

The main VI, Billard2025.vi, orchestrates the entire processing sequence. Data are exchanged through intermediate files, while stdout/stderr streams are captured when external programs are called.

3 Execution sequence

1. LabVIEW loads a PNG image from the sequence.
2. FindBillardBox.vi estimates the usable inner region of the table (BillardBox).
3. FrameToPosition.vi converts the frame into the binary file pixmap.bin.
4. LabVIEW launches Pix2Pos through System Exec, passing 29 parameters: the rectangle, colour ranges and BallSize.
5. Pix2Pos (C) reads pixmap.bin and writes pos.txt.
6. LabVIEW reads pos.txt and builds the trajectories X_r , Y_r , X_y , Y_y , X_w and Y_w .
7. MatlabScript.vi generates AnalyseTx.m, inserting the successive positions and parameters such as thresholds and cushion-contact colours.
8. RunMatlab.vi executes MATLAB to generate the ScoreSheet figure, export the PDF and create SummaryTx.txt.

3.1 Pipeline overview



4 C program Pix2Pos

4.1 Operating principle

- Reads pixmap.bin: width and height (uint32), followed by pixels in 0x00RRGGBB format.
- Builds red, yellow and white masks within the table rectangle using the RGB ranges.
- Uses integral images to score each BallSize × BallSize window efficiently, then retains the highest score for each colour.

4.2 Error handling

To allow LabVIEW to interpret the result easily:

- **Successful execution:** the program terminates with exit code = 0.
- **Blocking error:** the program writes a message to stderr in the form ERROR(<code>): ... and terminates with a negative exit code, as documented in the original version.
- **Warning only:** for example, too many pixels or fewer than three balls detected, the program writes WARNING: ... to stderr but keeps exit code = 0 so that the pipeline can continue.

4.3 Main cases handled and example output

- **Invalid arguments:** incorrect number of parameters (argc != 30) or a non-integer value (ERR_PARSE_* codes).
- **Input file:** pixmap.bin cannot be opened, the file is too small or a read is incomplete.
- **Dimensions:** width or height outside [100..1000], with an attempt to correct endianness using swap32.
- **Pixels:** too few pixels cause an error; too many pixels cause a warning, and excess pixels are ignored.
- **Geometry:** the table rectangle is invalid or inconsistent with the image.
- **Memory and output:** mask or integral-image allocation fails, or pos.txt cannot be written.
- **Partial detection:** fewer than three balls detected causes a warning; missing values are set to -1, -1, 0.

Examples written to stderr, with English translations of the original messages:

ERROR(-9): Not enough pixels in the file → exit = -9

WARNING: Too many pixels: excess pixels will be ignored → exit = 0

5 MATLAB trajectory analysis and output generation

5.1 Inputs and outputs

The MATLAB script is called from LabVIEW and primarily uses the following data.

Inputs: seqName; Xr, Yr, Xy, Yy, Xw and Yw, all of equal length; MoveDistPx; BallBorderDist; reboundColorWin and reboundColorLoss, specified as a colour letter or an [r g b] vector within [0..1]; and contextual parameters such as image height H = 480, ball size and configuration-dependent thresholds.

Outputs: generation of the ScoreSheetXX figure and PDF export, plus a SummaryXX.txt file in the prescribed format referred to in the original report.

5.2 Preprocessing for our C to MATLAB pipeline

In pos.txt, an undetected ball is encoded as -1, -1, 0. To make processing robust:

- The -1 values are replaced with NaN.
- Coordinates are converted to the MATLAB reference frame, reversing the vertical axis through the flip function: $y_mat = H - y_img$.
- NaN values are filled with the next valid value using `interp1(..., 'next')`, a function created in the MATLAB script. This models a stationary ball rather than creating artificial motion.
- Tracking outliers are removed using `isoutlier` with a moving median and replaced with the previous value. This stabilises the display, travelled-distance calculations and cushion-contact detection.

5.3 Ball order and the three cushion rule

The verdict is determined from the onset of movement:

- For each ball, we take the first index at which its distance from the initial position exceeds MoveDistPx. The first ball is the one with the smallest index; ties are resolved using `seglen1`. The second and third balls are then ordered by the onset of their movement.
- Cushion contacts of the first ball are detected when it is less than BallBorderDist from a frame boundary. Only contacts strictly between the movement onset of the second ball and that of the third ball are counted.
- The verdict is Win if the count is at least 3; otherwise it is Loss. Contact colours follow reboundColorWin or reboundColorLoss.

5.4 Error handling

The script includes explicit checks to avoid inconsistent output:

- **Trajectory consistency:** checks on type, non-emptiness and equal lengths, using `assertVectorSameLength` and an overall validation.
- **Invalid colours:** `validateMatlabColor` rejects values outside the accepted format, namely an allowed colour letter or RGB values within [0..1], and raises an explicit MATLAB error.
- **Edge cases:** if the second or third ball never moves, its index is infinite, the verdict is automatically Loss, and the script continues with a consistent display.
- **Summary writing:** if `Summary%s.txt` cannot be opened, a warning is issued.

6 LabVIEW

6.1 FindBillardBox.vi subVI

Purpose: automatically determine the bounding box of the table's dark-blue frame to obtain Xmin, Xmax, Ymin and Ymax. This rectangle, named Billard Box, is then used by the C program when searching for the balls.

Inputs and outputs

Direction	Name	Description
Input	Path	Path to the PNG image file
Input	Dark Blue Frame	Colour thresholds: Rmin/Rmax, Gmin/Gmax, Bmin/Bmax
Input	error in	Incoming LabVIEW error cluster
Output	Billard Box	Cluster {Xmin, Xmax, Ymin, Ymax}
Output	error out	Outgoing LabVIEW error cluster

Operating principle

- 1. Read the image:** the PNG file is read and converted to displayable Picture Control data, then to a two-dimensional array of pixels.
- 2. Scan the entire image:** two nested For loops visit every pixel, with y indexing rows and x indexing columns.
- 3. Test for the dark-blue frame:** extract R, G and B for each pixel and test the colour ranges below.
 $(R \in [Rmin, Rmax]) \wedge (G \in [Gmin, Gmax]) \wedge (B \in [Bmin, Bmax])$
The three conditions are combined using AND to produce the boolean isDarkBlueFrame.
- 4. Update the extrema using shift registers and Select:** for example, if isDarkBlueFrame is true and $x < Xmin$, set $Xmin := x$. Comparisons ($<$ or $>$) are combined using AND and passed to Select to choose between the current value and the new value.
- 5. Return the result:** at the end of the scan, the four values are assembled into the Billard Box cluster.

Register initialisation: the four shift registers are initialised to guarantee the first update. Xmin and Ymin are set to a large value approximating positive infinity; Xmax and Ymax are set to 0.

6.2 FrameToPosition.vi

Purpose: starting from a PNG frame, this VI reads and displays the image in a Picture Control; converts it to Pixmap.bin in the binary format expected by the C program; runs Pix2Pos using BillardBox, RGB thresholds and BallDiameter; and reads Pos.txt to extract the three balls' (x, y) positions and scores.

Inputs and outputs

Direction	Name	Description
Input	PNG Path	Path to frame Tx/00xxx.png
Input	BillardBox	Cluster {Xmin, Xmax, Ymin, Ymax}
Input	Red Range	Thresholds {Rmin, Rmax, Gmin, Gmax, Bmin, Bmax}
Input	Yellow Range	Thresholds {Rmin, Rmax, Gmin, Gmax, Bmin, Bmax}
Input	White Range	Thresholds {Rmin, Rmax, Gmin, Gmax, Bmin, Bmax}
Input	Blue Background Range	Thresholds {Rmin, Rmax, Gmin, Gmax, Bmin, Bmax}; project parameter
Input	BallDiameter	Diameter or square-window size used by Pix2Pos
Input	error in	Incoming LabVIEW error cluster
Output	PNG picture	Frame display in the Picture Control
Output	x/y/score red	Result for the red ball
Output	x/y/score yellow	Result for the yellow ball
Output	x/y/score white	Result for the white ball
Output	Message C / erreurs	Message returned by Pix2Pos execution through stderr/stdout
Output	error out	Outgoing LabVIEW error cluster

Operating principle

- 1. Read the PNG frame:** the file specified by PNG Path is read and converted to PNG picture for display.
- 2. Convert the image to Pixmap.bin.**
- 3. Prepare the Pix2Pos command line:** parameters are ordered and concatenated into a string of 29 command-line arguments for the C program.
- 4. Execute the C program:** call ./Pix2Pos using a system-execution node. The returned stdout/stderr message is captured in Message C / erreurs.
- 5. Read and parse Pos.txt:** the file is opened and converted to a string; values are extracted using Scan From String.

6.3 MatlabScript.vi

Purpose: this VI builds the MATLAB analysis script, AnalyseTx.m, using the previously extracted Pos.txt data and the LabVIEW configuration parameters. It creates a text file containing executable MATLAB instructions with the parameters inserted automatically. The file is saved in the working directory and executed by RunMatlab.vi.

Inputs and outputs

Direction	Name	Description
Input	Xr, Yr, Xy, Yy, Xw, Yw	Trajectory coordinates for the red, yellow and white balls
Input	reboundColorWin	Cushion-contact colour for a successful shot
Input	reboundColorLoss	Cushion-contact colour for an unsuccessful shot
Input	BallBorderDist	Distance from the table boundary used to detect contacts
Input	MoveDistPx	Displacement threshold in pixels for detecting ball movement
Input	BallminScore	Minimum acceptable score for a valid C detection
Input	Path MatlabScript	Full path of the AnalyseTx.m file to create
Input	error in	Incoming LabVIEW error cluster
Output	Script Path	Full path of the generated MATLAB file
Output	error out	Outgoing LabVIEW error cluster

Operating principle

- 1. Prepare the data:** read coordinate arrays Xr...Yw and convert them into MATLAB strings using sprintf, in the format [x1 x2 x3 ...].
- 2. Generate the text:** concatenate the MATLAB script line by line.
- 3. Write the file:** save the final text as Analyse<seqName>.m using Write to Text File.

6.4 LabVIEW error handling

Error-cluster propagation: we first connected the error wire to every subVI and main processing stage. This propagates an error detected anywhere in the diagram automatically.

Stopping processing on error: to avoid long or unnecessary tasks when an error is already present, we introduced state machines whose transitions depend on the inverse of the error status, NOT error.status.

Integrating Pix2Pos errors: the C program returns an explicit message through stderr. We capture it and, on failure, insert it into error.source using Select, with a non-zero status indicating an error.

7 Comments

Coordinate convention: we chose x pointing right and y pointing up. We adjust this order when building the Pix2Pos command line. This is an internal convention and does not affect the project results.

MATLAB Score Sheet: when this option is disabled, the MATLAB script is not generated and MATLAB is not launched. If no error is detected, the program deliberately writes an empty Summary file to prevent accidental reuse of the Summary from a previous run, as discussed in question #327.